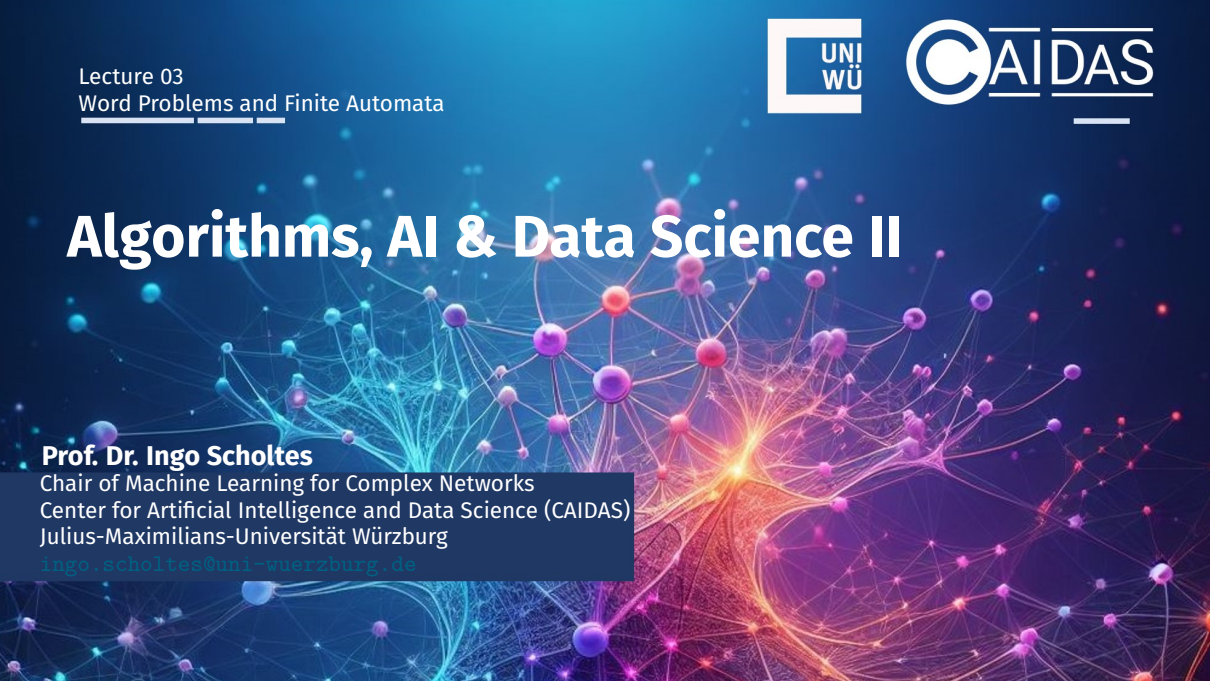
Lecture 03
Word Problems and Finite Automata

Algorithms, AI & Data Science II



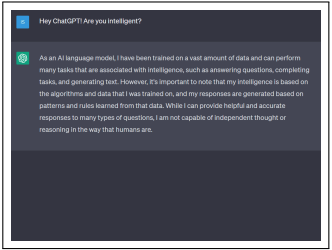
Prof. Dr. Ingo Scholtes
Chair of Machine Learning for Complex Networks
Center for Artificial Intelligence and Data Science (CAIDAS)
Julius-Maximilians-Universität Würzburg
ingo.scholtes@uni-wuerzburg.de

Notes:

- **L03: Word Problems and Finite Automata** 05.05.2025
- **Educational objective:** We continue our exploration of formal languages and grammars, introduce the Chomsky hierarchy and explore automata models for word problems.
  - Word Problems
  - Finite Automata
  - Pushdown Automata
  - Turing Machines
- **Handout version:** *2025/05/05 11:20:23*

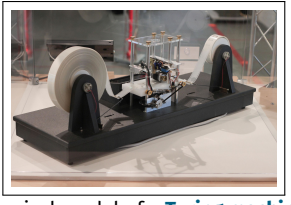
Motivation

- how can we **formally state a problem** such that a machine can given an answer?
- we introduced **mathematical foundations** of computer science
  1. sets and set operations
  2. relations and functions
- we introduced **formal languages** and used **formal grammars** to define the syntax of words
- how can we **algorithmically decide** whether a word belongs to a given language or not?
- how does the **difficulty of this decision problem** differ across different formal languages?



response from large language model ChatGPT

image credit: OpenAI screenshot, taken on 12/04/2023



physical model of a **Turing machine**

image credit: Rocky Acosta, Wikimedia Commons, CC-BY-SA

Notes:

Formal languages and grammars

For finite alphabet Σ, **formal language** $L \subseteq \Sigma^*$ is a possibly infinite set of words $w \in \Sigma^i$ (for $i = 0, 1, \ldots$)

example:

$$L = \{a^n b^m | n, m \geq 0\} \text{ over } \Sigma = \{a, b\}$$

For finite alphabet Σ, **formal grammar** $G = (V, \Sigma, P, S)$ is four-tuple with variables V, start symbol $S \in V$ and production rules P.

example:
$$P = \{S \to AB$$
$$A \to aA \mid \epsilon$$
$$B \to bB \mid \epsilon\}$$

$$S \to AB \to aAB \to aaAB \to aaB \to aab$$

- we can use grammars to define **syntax of words** that belong to given formal language
- grammar G produces word $w \in \Sigma^*$ iff production rules allow to substitute S by w, i.e. $S \to^* w$
- we say that $L(G)$ is the **formal language produced by grammar** G

Notes:

Chomsky hierarchy

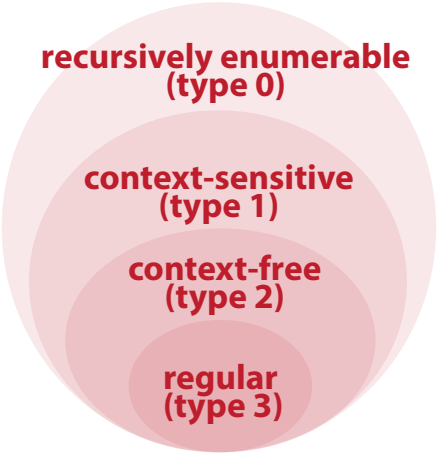
- we **categorize grammars into a hierarchy** based on constraints that must be satisfied by production rules

definition

Let $G = (V, \Sigma, P, S)$ be a grammar.

- Every grammar G is **recursively enumerable (type 0)**
- Grammar G is **context-sensitive (type 1)**, iff for all production rules $w_1 \to w_2 \in P : |w_1| \leq |w_2|$ (or rule has the form $S \to \epsilon$)
- Grammar G of type 1 is **context-free (type 2)**, iff for all rules $w_1 \to w_2 \in P : w_1 \in V$
- Grammar G of type 2 is **regular (type 3)**, iff for all rules $w_1 \to w_2 \in P : w_2 \in \Sigma \cup \Sigma V \cup \{\epsilon\}$

We say that a formal language $L \subseteq \Sigma^*$ is of type 0, 1, 2 or 3 iff a type 0, 1, 2 or 3 grammar G exists with $L(G)$, respectively.



Venn diagram of Chomsky hierarchy

- higher type = stricter rules = **less expressive**

Notes:

Context-free grammars and parsing trees

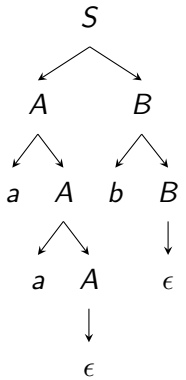
- consider words $a^n b^m$ where any number of a symbols is followed by any number of b symbols

grammar

$G = (\{A, B\}, \{a, b\}, P, S)$ with

$$P = \{S \to AB,$$
$$A \to aA \mid \epsilon,$$
$$B \to bB \mid \epsilon \}$$

- this grammar is **context-free but not regular**
- for context-free (and regular) grammars, production rules for word $w \in L(G)$ define **parsing tree**
  - leaf nodes = terminal symbols
  - internal nodes = variables
  - directed edges = variable substitutions
- parsing trees are **not necessarily unique**



parsing tree of word $aab \in L(G)$

Notes:

Group Exercise 03-01

1. Determine the type of the grammar from group exercise 02-02 in lecture L02.

   We have $|w_1| \leq |w_2|$ for all production rules $w_1 \to w_2$, i.e. this grammar is **context-sensitive**. E.g. due to the rule $aB \to ab$ this grammar is **not context-free**, i.e. it is a type 1 but not a type 2 (and thus not a type 3) grammar.

2. Try to construct a parsing tree for word $aaabbbccc$. What do you observe?

   Due to fact that the grammar is context-sensitive, it is impossible to construct a parsing tree, as we would need to substitute combinations of variables and terminal symbols, which violates the tree representation.

3. Can you write a regular grammar for the language $\{a^n b^m | n, m \geq 1\}$?

   $G = (\{A, B\}, \{a, b\}, P, S)$ with

$$P = \{S \to aA,$$
$$A \to aA \mid bB,$$
$$B \to bB \mid \epsilon \}$$

Notes:

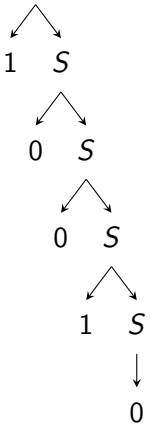
From context-free to regular grammars

- consider binary strings that represent **even numbers**

grammar

$G = (\{A, B\}, \{0, 1\}, P, S)$ with $P = \{S \to 0S \mid 1S \mid 0\}$

- what is the type of this grammar?
  - $|w_1| \leq |w_2|$, i.e. G is **context-sensitive (type 1)**
  - $w_1 \in V$, i.e. G is additionally **context-free (type 2)**
  - $w_2 \in \Sigma \cup \Sigma V$, i.e. G is additionally **regular (type 3)**
- for regular languages, parsing trees are **degenerate lists**



parsing tree of word $10010 \in L(G)$
(decimal 18)

Notes:

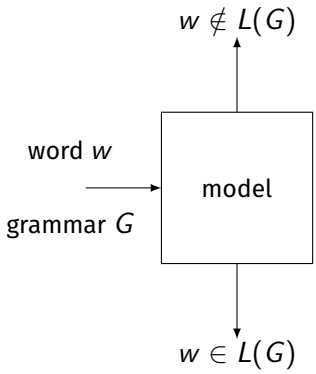
Word decision problems

- for grammar G and word $w \in \Sigma^*$, we call the question whether $w \in L(G)$ the **word decision problem**
- which computational model is needed to solve the word decision problem for a given grammar G?

exemplary problems

- check whether a sequence of characters $w \in \Sigma^*$ is a grammatically correct German sentence
- check whether a sequence of characters is a correct arithmetic expression
- check whether a file contains a valid C++ program

- depends on type (i.e. expressiveness) of grammar
- yields insights into **expressiveness of computational models** and limits of what can be computed/decided



Notes:

Finite automata

- consider **automaton** that consists of **finite memory** (states) and **transitions between states**

definition

Finite Automaton is a five-tupel $M = (Q, \Sigma, \delta, q_0, F)$ with

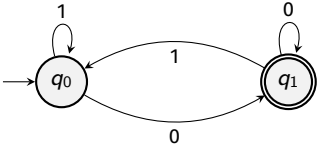
- Q a finite set of **states**, where $q_0 \in Q$ is the initial state and $F \subseteq Q$ is a set of **accepting (final) states**
- Σ is a finite **alphabet of input symbols**
- $\delta \subseteq Q \times \Sigma \times Q$ is a **transition relation**, where for $(q, \sigma, q') \in \delta$ q' is the next state to which the automaton transitions if it reads σ in state q

- M is a **deterministic** finite automaton (DFA) iff δ is functional, i.e. for every combination of current state and input symbol there is only one possible next state
- we can **use graph $G = (V, E)$ to represent automaton M**, where nodes $V = Q$ represent states and (labeled) edges $E \subseteq V \times V$ represent transitions

M with $Q = \{q_0, q_1\}$, $F = \{q_1\}$ and

$$\delta = \{(q_0, 0, q_1), (q_0, 1, q_0), (q_1, 0, q_1), (q_1, 1, q_0)\}$$

M is **deterministic** finite automaton (DFA)



graph representation of M

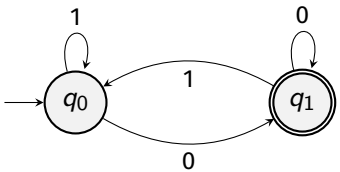
Notes:

DFAs and formal languages

- we say that **DFA M accepts word** $w = w_1 w_2 \dots w_n \in \Sigma^*$ iff M, started in q_0 and processing inputs $w_1, \dots, w_n$ reaches final state $q_n \in F$, i.e.

$$\delta^*(q_0, w) := \delta(\dots \delta(\delta(q_0, w_1), w_2), \dots) \in F$$

- we call $L(M) = \{w \in \Sigma^* | \delta^*(q_0, w) \in F\}$ the **language recognized by** M
- for which languages is DFA sufficient to solve word decision problem?



example

$M = (\{q_0, q_1\}, \Sigma = \{0, 1\}, \delta, q_0, F = \{q_1\})$ accepts binary strings that correspond to even decimal numbers

Notes:

DFAs and regular languages

- the following theorem highlights an important relationship between DFAs and regular languages

  → MO Rabin, D Scott, 1959

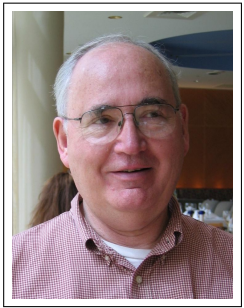
Rabin-Scott theorem

The set of languages that can be recognized by a deterministic finite automaton (DFA) is exactly the set of regular languages.

proof idea

1. use transition rules and states of DFA to create a grammar G that fulfils conditions of a regular grammar
2. show that any regular grammar G defines a **non-deterministic** finite automaton (NFA) M with $L(G) = L(M)$
3. show that for any NFA M, we can create DFA M' with $L(M) = L(M')$ $\square$

   → exercise sheet



Dana C. Scott, born 1932

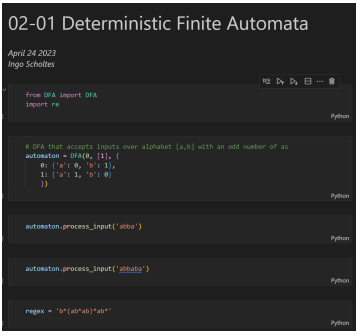
Turing Award 1976

image credit: Wikimedia Commons, Andrej Bauer, CC-BY-SA

Notes:

Practice Session

- we implement **deterministic finite automata** in python
- we use DFAs to **accept words** from regular languages
- we introduce **regular expressions**



practice session

see notebook 03-01 in `gitlab` repository at

→ https://gitlab.informatik.uni-wuerzburg.de/ml4nets_notebooks/2025_sose_akids2

Notes:

Context-free languages and finite automata?

- consider grammar G that produces **correctly bracketed arithmetic expressions** (with single variable a)
- G is **context-free (type 2)** since for all production rules $w_1 \to w_2$ we have $|w_1| \leq |w_2|$ and $w_1 \in V$
- G is **not regular (type 3)** since right side of production rules contains $S + T$, $T * F$ and (S)
- why can we not use a DFA to recognize this language?
- nesting of brackets in expressions can **be of arbitrary depth**, i.e. **finite memory of DFA is not sufficient**
- **finite memory** of DFA limits their expressive power

grammar

$G = (V, \Sigma, P, S)$ with
$V = \{S, T, F\}, \Sigma = \{a, +, *, (,)\}$ and

$$P = \{ \begin{array}{l} S \to T \mid S + T, \\ T \to F \mid T * F, \\ F \to a \mid (S) \} \end{array}$$

Notes:

Pushdown automata (PDA)

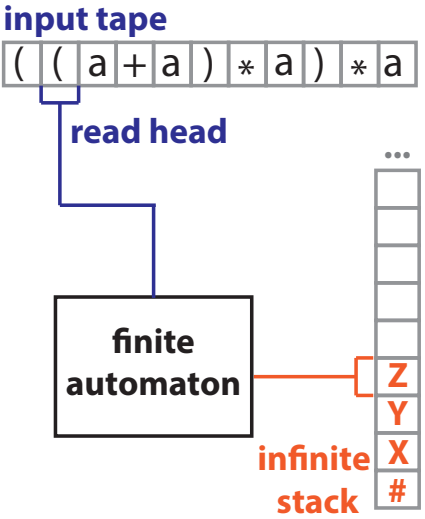
- idea: extend automaton by a stack, i.e. **infinite memory** that can be accessed in Last-In First-Out (LIFO) fashion

definition

A **pushdown automaton** (PDA) is a six-tupel $M = (Q, \Sigma, \Gamma, \delta, q_0, \#)$

- Q a finite set of **states**, where $q_0 \in Q$ is initial state
- Σ a finite alphabet of **input symbols**
- Γ a finite alphabet of **stack symbols**, where $\#$ is initial (bottom-most) symbol
- $\delta \subseteq Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \times Q \times \Gamma^*$ a **transition relation**, where for $(q, x, y, q', w) \in \delta$ automaton in state q reading input symbol x, replaces top-most stack symbol $y \in \Gamma$ by $w \in \Gamma^*$ and transitions to state $q' \in Q$

- transition relation δ allows **transitions between states without reading input symbol** (i.e. reading ϵ)
- M is **deterministic** PDA (DPDA) iff δ is functional, i.e. $\delta : Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \to Q \times \Gamma^*$

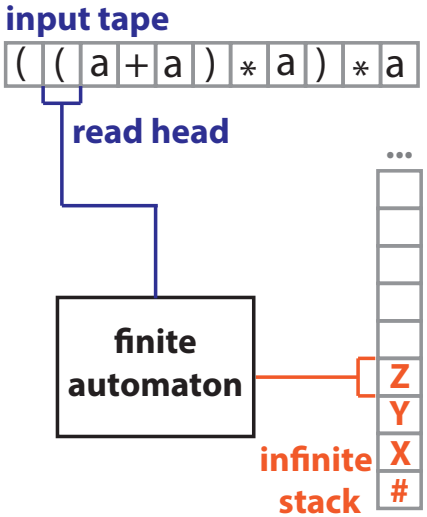


pushdown automaton

Notes:

PDAs and languages

- we use **graph to represent transition relation of PDA**, where (q, x, y, q', w) is represented by edge $(q, q') \in E$ with label $x, y|w$
- for PDA M we call $(q, w, s) \in Q \times \Sigma^* \times \Gamma^*$ a **configuration of M**, where
  - q is current state,
  - w is remaining symbols on input tape, and
  - s is current stack contents
- transition relation δ defines **transitions between configurations**, i.e. $c_0 \to c_1 \to \ldots \to c_n$
- M accepts word $w \in \Sigma^*$ iff **stack is empty** after input has been processed, i.e. M reaches config. $(q, \epsilon, \#)$
- we define **language $L(M)$ recognized by M** as $L(M) = \{w \in \Sigma^* | (q_0, w, \#) \to^* (q, \epsilon, \#) \text{ for } q \in Q\}$

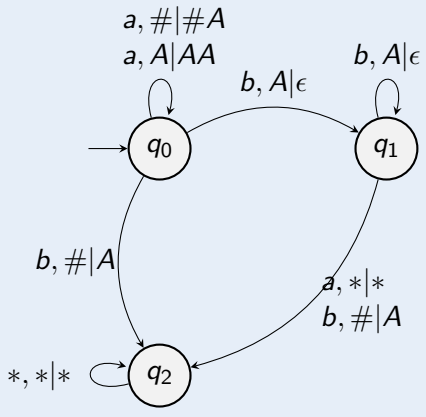


pushdown automaton

Notes:

Group exercise 03-02

- Define a pushdown automaton that accepts words from the formal language $L = \{a^n b^n | n \geq 1\}$. Compute the sequence of configurations that your automaton generates while processing the input *aaabbb*.



input	configuration
-	$(q_0, aaabbb, \#)$
a	$(q_0, aabbb, \#A)$
a	$(q_0, abbb, \#AA)$
a	$(q_0, bbb, \#AAA)$
b	$(q_1, bb, \#AA)$
b	$(q_1, b, \#A)$
b	$(q_1, \epsilon, \#)$

Notes:

PDAs and context-free languages

- for context-free language $L = \{a^n b^n | n \geq 1\}$, we found **deterministic PDA** M with $L(M) = L$
- PDA with deterministic transition function accepts subset of **deterministic context-free languages**
- consider context-free language consisting of all **even-length palindromes** $L = \{ww^R | w \in \Sigma^*\}$ with $\Sigma = \{a, b\}$
- requires PDA with **non-deterministic transition relation** δ that allows to "guess" center of word

theorem

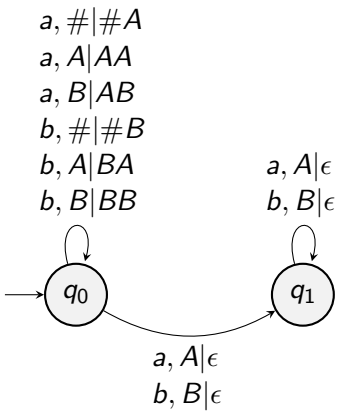
The set of languages that can be recognized by a **non-deterministic PDA** is exactly the set of **context-free (type 2)** languages.

proof idea

1. starting from context-free grammar, define NPDA that simulates production rules
2. starting from NPDA, define context-free grammar G with production rules that capture transition relations

context-free language

$L(M) = \{ww^R | w \in \Sigma^*\}$

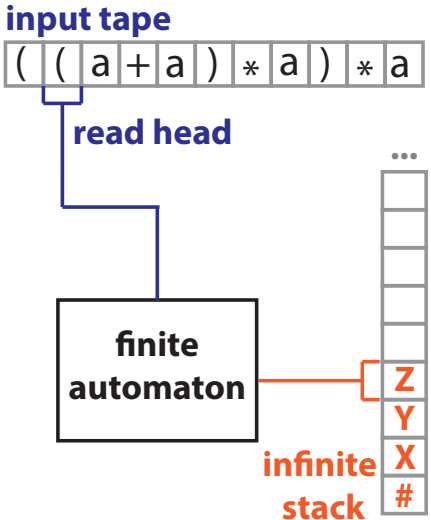


Notes:

- Note the two non-deterministic transitions in q_0 for input symbols a and stack symbol A as well as for input symbol b and stack symbol B. By non-deterministically transitioning to q_1 the automaton “guesses” the center of the palindrom.

In summary …

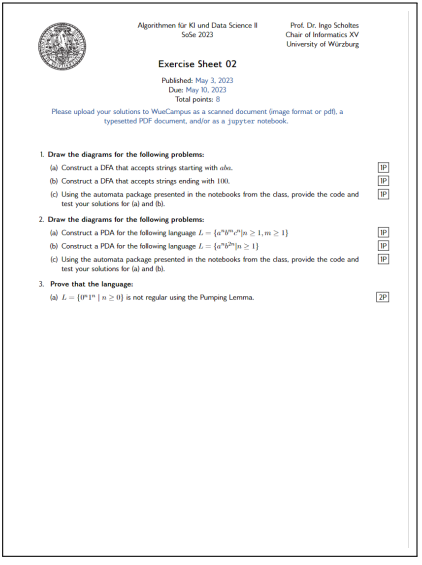
- we have covered **formal languages and grammars**
- we can use different **automata modles** to address the **word decision problem**
- different Chomsky types of languages require different automata models
- formal languages and automata allow us to formalize **which problems are decidable**
- basis to study concepts like **“artificial intelligence”** and answer question **whether machines can “think”**



Notes:

Exercise sheet 01

- **first exercise sheet** will be released today
- solutions are due **May 12th** (via WueCampus)
- present your solution to earn bonus points



Algorithmen für KI und Data Science II
SoSe 2023

Prof. Dr. Ingo Scholtes
Chair of Informatics XV
University of Würzburg

Exercise Sheet 02

Published: May 3, 2023
Due: May 10, 2023
Total points: 8

Please upload your solutions to WueCampus as a scanned document (image format or pdf), a typesetted PDF document, and/or as a `jupyter` notebook.

1. **Draw the diagrams for the following problems:**
   (a) Construct a DFA that accepts strings starting with *abc*. [1P]
   (b) Construct a DFA that accepts strings ending with 100. [1P]
   (c) Using the automata package presented in the notebooks from the class, provide the code and test your solutions for (a) and (b). [1P]
2. **Draw the diagrams for the following problems:**
   (a) Construct a PDA for the following language $L = \{a^n b^m c^n | n \geq 1, m \geq 1\}$ [1P]
   (b) Construct a PDA for the following language $L = \{a^n b^{2n} | n \geq 1\}$ [1P]
   (c) Using the automata package presented in the notebooks from the class, provide the code and test your solutions for (a) and (b). [1P]
3. **Prove that the language:**
   (a) $L = \{0^n 1^n \mid n \geq 0\}$ is not regular using the Pumping Lemma. [2P]

Notes:

Self-study questions

1. What is the difference between a content-sensitive and context-free language?
2. Give an example for a language that is regular.
3. Give an example for a language that is context-free but not regular.
4. Consider the definition of a non-deterministic finite automaton (NFA), which is identical to a DFA except for the fact that δ is not functional. Give an example for such an automaton.
5. Give a constructive proof that for every regular language L there is a DFA M with $L(M) = L$.
6. Give an example for a language for which the word problem can be decided by a non-deterministic PDA but not by a deterministic PDA.
7. Explain why the word problem for the language consisting of bracketed arithmetic expressions can not be decided by a DFA.
8. Show that any regular grammar defines a non-deterministic finite automaton.
9. Give an example for a context-free grammar for which the production rules can not be simulated by a deterministic PDA.
10. Consider the (context-sensitive) language $L = \{a^n b^n c^n | n \geq 1\}$. Argue whether there exists a non-deterministic or deterministic PDA that recognizes this language.

Notes:

References

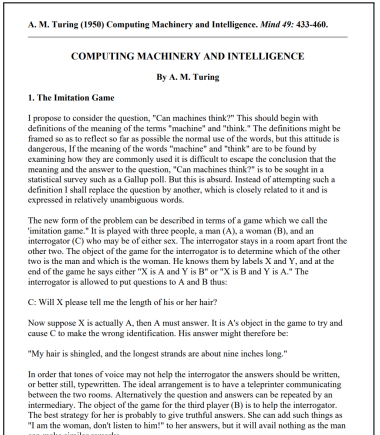
reading list

- N Chomsky: **Three models for the description of language**, 1956
- MO Rabin and D Scott: **Finite automata and their decision problems**, IBM J. Res. Dev., 1959
- Y Bar-Hillel, M Perles, E Shamir: **On formal properties of simple phrase-structure grammars**, Zeitschrift für Phonetik, Sprachwissenschaft, und Kommunikationsforschung, 1961
- S-Y Kuroda: **Classes of languages and linear-bounded automata**, 1964
- D Knuth: **The Art of Computer Programming. Vol. 1: Fundamental Algorithms**, Addison–Wesley, 1973

acknowledgments

some examples are based on the following books and lectures:

- U Schöning: **Theoretische Informatik - kurzgefasst**, Spektrum Akadem. Verlag, 2001
- J Esparza: **Automata theory: An algorithmic approach**, lecture notes, 2020



A. M. Turing (1950) Computing Machinery and Intelligence. *Mind 49:* 433-460.

COMPUTING MACHINERY AND INTELLIGENCE

By A. M. Turing

1. The Imitation Game

I propose to consider the question, "Can machines think?" This should begin with definitions of the meaning of the terms "machine" and "think." The definitions might be framed so as to reflect so far as possible the normal use of the words, but this attitude is dangerous, If the meaning of the words "machine" and "think" are to be found by examining how they are commonly used it is difficult to escape the conclusion that the meaning and the answer to the question, "Can machines think?" is to be sought in a statistical survey such as a Gallup poll. But this is absurd. Instead of attempting such a definition I shall replace the question by another, which is closely related to it and is expressed in relatively unambiguous words.

The new form of the problem can be described in terms of a game which we call the 'imitation game." It is played with three people, a man (A), a woman (B), and an interrogator (C) who may be of either sex. The interrogator stays in a room apart front the other two. The object of the game for the interrogator is to determine which of the other two is the man and which is the woman. He knows them by labels X and Y, and at the end of the game he says either "X is A and Y is B" or "X is B and Y is A." The interrogator is allowed to put questions to A and B thus:

C: Will X please tell me the length of his or her hair?

Now suppose X is actually A, then A must answer. It is A's object in the game to try and cause C to make the wrong identification. His answer might therefore be:

"My hair is shingled, and the longest strands are about nine inches long."

In order that tones of voice may not help the interrogator the answers should be written, or better still, typewritten. The ideal arrangement is to have a teleprinter communicating between the two rooms. Alternatively the question and answers can be repeated by an intermediary. The object of the game for the third player (B) is to help the interrogator. The best strategy for her is probably to give truthful answers. She can add such things as "I am the woman, don't listen to him!" to her answers, but it will avail nothing as the man can make similar remarks.

We now ask the question, "What will happen when a machine takes the part of A in this game?" Will the interrogator decide wrongly as often when the game is played like this as he does when the game is played between a man and a woman? These questions replace our original, "Can machines think?"

Notes: